# MFA (Telegram 2FA) — Architecture & Control Flow

This document explains how **Telegram-based MFA/2FA** is wired in the MyTonWallet client, with **code-symbol + line references** for both:

- **MFA disabled** (normal signing + immediate broadcast)
- **MFA enabled** (sign → create MFA request → confirm in Telegram → backend broadcasts → client polls for confirmation)

> *Scope note: MFA support in this codebase is currently **TON-only** (it's stored under `byChain.ton.mfa`) and is tightly coupled to **Wallet V5R1 extensions**.*

---

## Key Concepts (What "MFA enabled" means)

When MFA is enabled for a TON wallet, the client does **not** broadcast a transfer immediately. Instead it:

1. Signs an **MFA Extension request** (instead of a direct wallet external message).
2. Publishes that request to an **MFA API service**.
3. Redirects the user to a **Telegram bot / mini app** to confirm.
4. Polls the MFA API until it becomes **confirmed**.

The presence of MFA is stored on the account as:

- `ApiTonWallet.mfa` in storage: `src/api/types/storage.ts#L13` (field at `#L17`)
- Persisted via `setAccountExtensionAddress(...)`: `src/api/methods/auth.ts#L455`

---

## Architecture (Pieces & Responsibilities)

### Client layers (UI → Global actions → API worker → TON chain)

- UI triggers actions and renders states.
  - Transfer MFA confirmation UI: `src/components/transfer/TransferConfirmMfa.tsx#L24`
  - Generic MFA confirmation UI: `src/components/common/MfaConfirm.tsx#L39`
- Global actions coordinate UI state and call the SDK (`callApi`).
  - Transfer submit + state transitions: `src/global/actions/api/transfer.ts#L227`
  - Polling MFA status: `src/global/actions/api/transfer.ts#L392`
  - MFA install/remove actions: `src/global/actions/api/mfa.ts#L8`, `src/global/actions/api/mfa.ts#L56`
  - Install request creation (opens Telegram): `src/global/actions/api/settings.ts#L8`
- SDK runs behind `callApi` in a Worker (Classic platforms).
  - Worker connector `callApi(...)`: `src/api/providers/worker/connector.ts#L58`
- TON chain implementation signs/broadcasts or produces `mfaRequest`.
  - Gasfull transfers: `src/api/chains/ton/transfer.ts#L395`
  - MFA signing decision: `src/api/chains/ton/transfer.ts#L1080` (see MFA branch at `#L1126`)

### On-chain (TON contracts)

- **Wallet V5R1** with extensions enabled (MFA uses "extension auth" requests).

    - The install flow explicitly requires `WalletContractV5R1`: `src/api/chains/ton/mfa.ts#L34`
- **MfaExtension** contract wrapper + request body format:
    - Opcodes: `src/api/chains/ton/contracts/MfaExtension.ts#L17`
    - Request payload builder: `src/api/chains/ton/contracts/MfaExtension.ts#L163`
- **MfaMaster** contract used for fee estimation:
    - Fee method wrapper: `src/api/chains/ton/contracts/util.ts#L57`

### Off-chain (Required services when MFA is enabled)

This repo expects two external pieces (configured via env vars):

1. **MFA API** (HTTP) at `MFA_API_BASE_URL` used by:
    - `createMfaRequest(...)`: `src/api/common/mfa.ts#L28`
    - `getMfaRequest(...)`: `src/api/common/mfa.ts#L58`
    - `createInstallMfaRequest(...)`: `src/api/common/mfa.ts#L46`
    - `getInstallMfaRequest(...)`: `src/api/common/mfa.ts#L64`
2. **Telegram bot / mini app** URL at `MFA_BOT_URL` opened by:
    - `MfaConfirm` "Confirm" button: `src/components/common/MfaConfirm.tsx#L49`
    - Install request opener: `src/global/actions/api/settings.ts#L16`
    - Remove request opener: `src/global/actions/api/mfa.ts#L66`

**Do we need to deploy additional backend services?**

- **If MFA is disabled**: no extra services beyond the usual TON API providers (`TONCENTER_*`, `TONAPIIO_*`, etc.).
- **If MFA is enabled**: yes — the client requires an **MFA API** + a **Telegram bot/mini app** that can confirm requests and mark them as confirmed.
    - This repository only contains the client-side integration; the MFA service implementation is **not** present here.

---

## State Machine (Where MFA appears in UI)

Transfers use `TransferState.ConfirmMfa`:

- Enum: `src/global/types.ts#L198` (value at `TransferState.ConfirmMfa` is `#L205`)
- Transition into MFA confirmation after submit: `src/global/actions/api/transfer.ts#L315`

MFA confirmation modal polls once per second:

- `useInterval(... updateMfaRequestStatus ...)`: `src/components/transfer/TransferConfirmMfa.tsx#L37`
- Poll action handler: `src/global/actions/api/transfer.ts#L392`

---

## Control Flow — Transfer with MFA disabled

1. UI submits transfer:
    - `addActionHandler('submitTransfer', ...)`: `src/global/actions/api/transfer.ts#L227`
2. Action calls SDK:

- `callApi('submitTransfer', chain, options)`: `src/global/actions/api/transfer.ts#L308`

3. SDK method routes to chain implementation:
   - `submitTransfer(...)`: `src/api/methods/transfer.ts#L131`
   - Calls `chains[chain].submitGasfullTransfer(...)`: `src/api/methods/transfer.ts#L168`
4. TON chain signs an **external** wallet message and broadcasts it:
   - `submitGasfullTransfer(...)`: `src/api/chains/ton/transfer.ts#L395`
   - `signTransaction(...)` produces `transaction` (no `mfaRequest`): `src/api/chains/ton/transfer.ts#L463`
   - `sendExternal(...)` sends BOC: `src/api/chains/ton/transfer.ts#L511`
5. SDK creates a local activity and returns `activityId`:
   - `createLocalTransactions(...)`: `src/api/methods/transfer.ts#L193`
6. Global marks the transfer complete:
   - Sets `TransferState.Complete`: `src/global/actions/api/transfer.ts#L315`

---

## Control Flow — Transfer with MFA **enabled**

The same entry points are used, but the TON chain returns an MFA request instead of broadcasting.

1. UI submits transfer:
   - `src/global/actions/api/transfer.ts#L227`
2. SDK method calls TON chain `submitGasfullTransfer`:
   - `src/api/methods/transfer.ts#L131`
3. TON chain detects MFA extension and fetches its seqno:
   - `mfaExtensionSeqno = await getMfaExtensionSeqno(...)`: `src/api/chains/ton/transfer.ts#L458`
4. TON chain signs the transaction *for the MFA Extension*:
   - `signTransaction({ ..., mfaExtensionSeqno })`: `src/api/chains/ton/transfer.ts#L463`
   - The signer path switches to MFA signing here:
     - `signTransactions(...)`: `src/api/chains/ton/transfer.ts#L1080`
     - MFA branch: `src/api/chains/ton/transfer.ts#L1126`
     - Calls `signer.signMfaTransactions(...)`: `src/api/chains/ton/transfer.ts#L1129`
     - `SignedMfaRequest` type: `src/api/chains/ton/util/signer.ts#L30`
5. TON chain returns `{ mfaRequest }` and **does not** send anything on-chain:
   - `if (mfaRequest) return { mfaRequest };`: `src/api/chains/ton/transfer.ts#L483`
6. SDK publishes the MFA request to the MFA API:
   - `if (result.mfaRequest) { ... createMfaRequest(...) }`: `src/api/methods/transfer.ts#L175`
   - API call: `src/api/common/mfa.ts#L28`
   - Result returned to UI contains `mfaRequestHash`: `src/api/methods/transfer.ts#L185`
7. Global switches UI into `TransferState.ConfirmMfa`:

- `state: ('mfaRequestHash' in result) ? TransferState.ConfirmMfa : ...` : `src/global/actions/api/transfer.ts#L315`

8. User is asked to confirm in Telegram:
   - `MfaConfirm.handleSubmit()` opens `MFA_BOT_URL?startapp=<hash>` : `src/components/common/MfaConfirm.tsx#L49`
9. Client polls MFA API until confirmation:
   - `updateMfaRequestStatus` : `src/global/actions/api/transfer.ts#L392`
   - Fetch method: `fetchMfaRequest(...)` : `src/api/methods/mfa.ts#L9`
   - HTTP call: `getMfaRequest(...)` : `src/api/common/mfa.ts#L58`
10. Once `isConfirmed` is true, UI completes:
    - `if (result?.isConfirmed) ... TransferState.Complete` : `src/global/actions/api/transfer.ts#L398`

---

## What exactly is in `SignedMfaRequest`?

`SignedMfaRequest` is produced by the `Signer` implementation:

- Type definition: `src/api/chains/ton/util/signer.ts#L30`
- Signing implementation (mnemonic/view/mock signer path):
  - `signMfaTransactionsWithPrivateKey(...)` : `src/api/chains/ton/util/signer.ts#L314`
  - Constructs the extension payload via `getBodyFromRequest(...)` : `src/api/chains/ton/contracts/MfaExtension.ts#L163`

The client only publishes `payload` + `signature` to the MFA API:

- `createMfaRequest({ walletAddress, payload: payload.toBoc(), signature })` : `src/api/methods/transfer.ts#L179`

> *Backend note: the exact backend flow is outside this repo. From the client contract it's clear the service must be able to:*
>
> - *store requests by `reqId`,*
> - *return `isConfirmed`,*
> - *and (for real end-to-end) broadcast the corresponding on-chain message and expose a `txHash` (returned by `getMfaRequest` at `src/api/common/mfa.ts#L11`).*

---

## MFA Install / Remove (Enable / Disable) Flows

### Enable MFA (install extension)

1. User starts install from Settings:
   - `createInstallMfaRequest` : `src/global/actions/api/settings.ts#L8`
2. SDK publishes an install request to the MFA API:
   - `publishInstallMfaRequest(...)` : `src/api/methods/mfa.ts#L17`
   - `createInstallMfaRequest(...)` : `src/api/common/mfa.ts#L46`
3. Client opens Telegram bot with an install marker:
   - `startapp = i-<reqId>` : `src/global/actions/api/settings.ts#L17`

4. Client polls install request until user info appears:
   - `updateInstallMfaRequest` : `src/global/actions/api/mfa.ts#L8`
   - `fetchInstallMfaRequest(...)` : `src/api/methods/mfa.ts#L13`
5. Once a Telegram user is linked, client deploys/installs the on-chain extension:
   - `installMfaFromRequest(...)` : `src/api/methods/mfa.ts#L24`
   - On-chain install: `installMfaExtension(...)` : `src/api/chains/ton/mfa.ts#L34`
6. Client persists the extension address in storage:
   - `setAccountExtensionAddress(...)` : `src/api/methods/auth.ts#L455`

### Disable MFA (remove extension)

1. User starts removal:
   - `submitRemoveMfa` : `src/global/actions/api/mfa.ts#L56`
2. SDK creates a removal payload and publishes it as an MFA request:
   - `publishRemoveMfaRequest(...)` : `src/api/methods/mfa.ts#L39`
   - Removal signing: `createRemoveMfaExtensionPayload(...)` : `src/api/chains/ton/mfa.ts#L14`
3. Client opens Telegram bot to confirm removal:
   - `startapp=<reqId>` : `src/global/actions/api/mfa.ts#L66`
4. Client polls until the request is confirmed, then clears MFA from account:
   - `updateRemoveMfaRequest` : `src/global/actions/api/mfa.ts#L71`
   - Clears `account.byChain.ton.mfa` : `src/global/actions/api/mfa.ts#L83`

---

## Known Limitations (current code behavior)

- Ledger accounts don't support MFA signing/install/remove in this implementation (methods throw):
  - `LedgerSigner.signMfaTransactions` : `src/api/chains/ton/util/signer.ts#L236`
  - `LedgerSigner.signInstallMfaRequest` : `src/api/chains/ton/util/signer.ts#L240`
  - `LedgerSigner.signRemoveMfaRequest` : `src/api/chains/ton/util/signer.ts#L244`
- Fee estimation draft path bails out when MFA is installed:
  - `// NOTICE: unable to calculate fees if 2FA enabled` : `src/api/chains/ton/transfer.ts#L229`
- Some flows treat `mfaRequest` as "unexpected" and effectively don't support MFA yet:
  - Staking wrappers: `src/api/methods/staking.ts#L52`
  - Multi-transfer: `src/api/chains/ton/transfer.ts#L924`

---

## Local Testing Setup

### 1) Run the client (no MFA)

```
cp .env.example .env
npm ci
npm run dev
```

### 2) Enable MFA features (client-side)

Add these env vars to `.env`:

```
MFA_BOT_URL="https://t.me/<your_bot>?startapp="
MFA_API_BASE_URL="http://localhost:4000"
MFA_MASTER_ADDRESS="<ton address>"
MFA_EXTENSION_CODE_HASH="<hex>"
```

Notes:

- The client reads these from `src/config.ts#L773`.
- MFA signing/install/remove currently requires a private-key-based signer (mnemonic); Ledger accounts are not supported (see `src/api/chains/ton/util/signer.ts#L236`).
- `webpack.config.ts` includes `MFA_API_BASE_URL` in CSP `connect-src` hosts (so browser fetches to the MFA API are allowed): `webpack.config.ts#L99` (directive at `webpack.config.ts#L117`).

### 3) Provide an MFA API + Telegram bot

To test end-to-end MFA you need:

- An **MFA API** implementing at minimum:
  - `POST /transaction` → `{ reqId }` (`src/api/common/mfa.ts#L28`)
  - `GET /transaction/:reqId` → `{ isConfirmed, txHash, ... }` (`src/api/common/mfa.ts#L58`)
  - `POST /installRequest` → `{ reqId }` (`src/api/common/mfa.ts#L46`)
  - `GET /installRequest/:reqId` → `{ user }` (`src/api/common/mfa.ts#L64`)
- A **Telegram bot / mini app** (at `MFA_BOT_URL`) that can:
  - show the pending request by `reqId`,
  - confirm it,
  - and trigger the backend to mark it confirmed (and broadcast on-chain if applicable).

If you don't have the real services, you can still test the **client UI state machine** by using a mock MFA API that:

- immediately returns a `reqId`,
- returns `isConfirmed=true` after a delay,
- and returns a dummy `user` for install requests.

---

## Quick "Does it work?" checklist

- Transfer without MFA:
  - No `TransferState.ConfirmMfa` and no Telegram redirect.
- Transfer with MFA:
  - `submitTransfer` returns `mfaRequestHash` (`src/api/methods/transfer.ts#L185`)
  - UI enters `TransferState.ConfirmMfa` (`src/global/actions/api/transfer.ts#L315`)
  - "Confirm" opens Telegram (`src/components/common/MfaConfirm.tsx#L49`)

- Polling completes when API returns `isConfirmed=true` ( `src/global/actions/api/transfer.ts#L398` )